

Lecture: 0011

Combinational Building Blocks

Two Philosophies for a CPU's Language

*The instruction set is the contract between hardware and software —
and there are two great ways to write it.*

CSA222: Modern Computer Architecture

Where We Left Off

From single gates to reusable blocks — L1 & L2 gave us the raw materials.



Gates & truth tables

AND, OR, NOT, XOR — and how any behaviour becomes a truth table (L1).



SOP / POS forms

Any truth table → a Boolean expression we can build in hardware (L2).



K-map minimisation

Fewest gates for a given function — area, power, delay all matter (L2).



NAND is universal

One gate builds everything. Today we build **USEFUL** things from gates.

Today: stop thinking in individual gates. Start thinking in blocks you reuse everywhere.

Today's Roadmap

Four building blocks — then we snap them together into a CPU's core.

1

Multiplexer/Selectors

Multiplexers pick one of many inputs; demultiplexers route one input to many outputs.



2

Decoders

Turn an n -bit code into one active line out of 2^n — the heart of addressing.



3

Adders

Half and full adders — the arithmetic every ALU is built on.

Then — the payoff: we show exactly where MUXes, decoders, and adders live inside an ALU and a register file.

Why Reusable Blocks?

Nobody designs a CPU one gate at a time — they use a parts catalogue.

~5

core block types

MUX, decoder, adder, register,
and a few more — repeated
millions
of times — build an entire
processor.



ABSTRACTION

Think 'select this input', not '17 gates wired thus'.



REUSE

One verified MUX design,
dropped in everywhere it's
needed.



VERIFICATION

Test the block once; trust
every copy of it.



SCALE

Small blocks cascade into big
ones — 2:1 MUX → 4:1 → 8:1.

1. Multiplexers

A data selector: many inputs go in, a select code picks exactly one to come out.

1. Multiplexers

The fundamental routing block of digital computer systems.

CORE DEFINITION

A Data Selector

A multiplexer acts as a digital traffic controller for processing signals.

Many inputs go in, but a dynamic select code picks exactly one to come out.



Many Inputs

Multiple source channels arrive carrying unique parallel data streams.



The Select Code

Control signals act as the selector, dynamically determining the routing path.

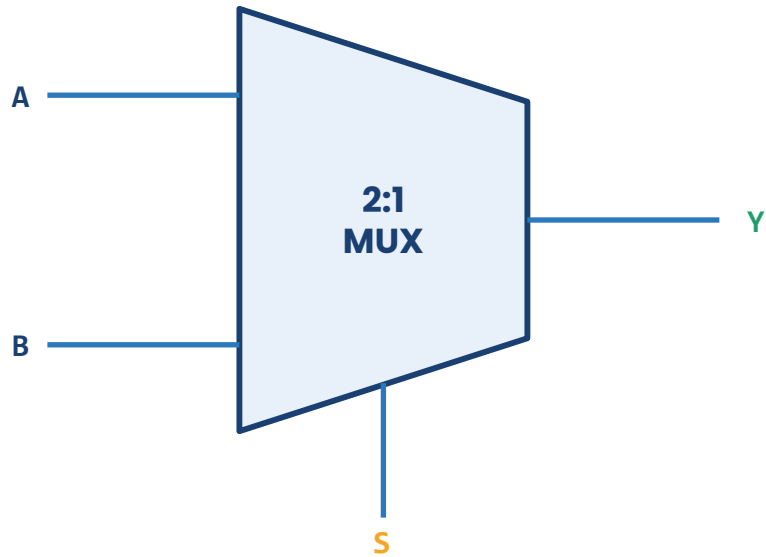


Exactly One Output

Only the chosen input line is connected and successfully passes outward.

The 2:1 Multiplexer

One select line chooses between two inputs — the simplest selector.



$S = 0 \rightarrow Y = A$ $S = 1 \rightarrow Y = B$

BOOLEAN EQUATION

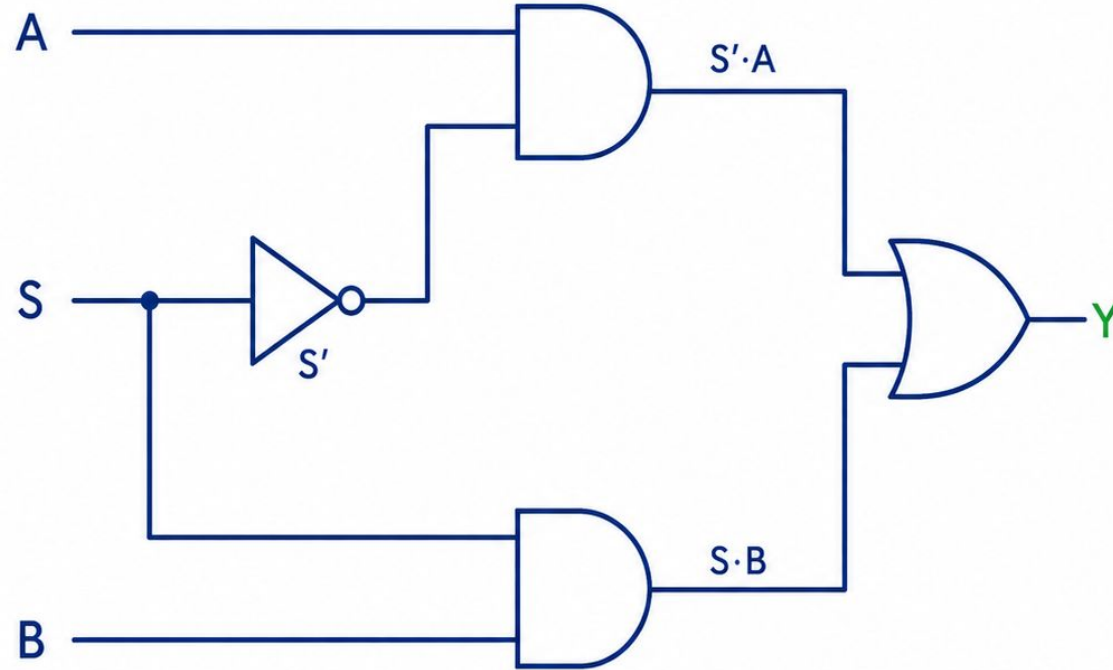
$$Y = S' \cdot A + S \cdot B$$

THINK: RAILWAY SWITCH

Two tracks (A, B) converge to one line (Y). The select signal S is the switch lever deciding which train passes through. Change S, and a completely different input flows out — instantly, no data is stored.

Building a 2:1 MUX from Gates

$Y = S' \cdot A + S \cdot B$ — *two ANDs, one NOT, one OR.*

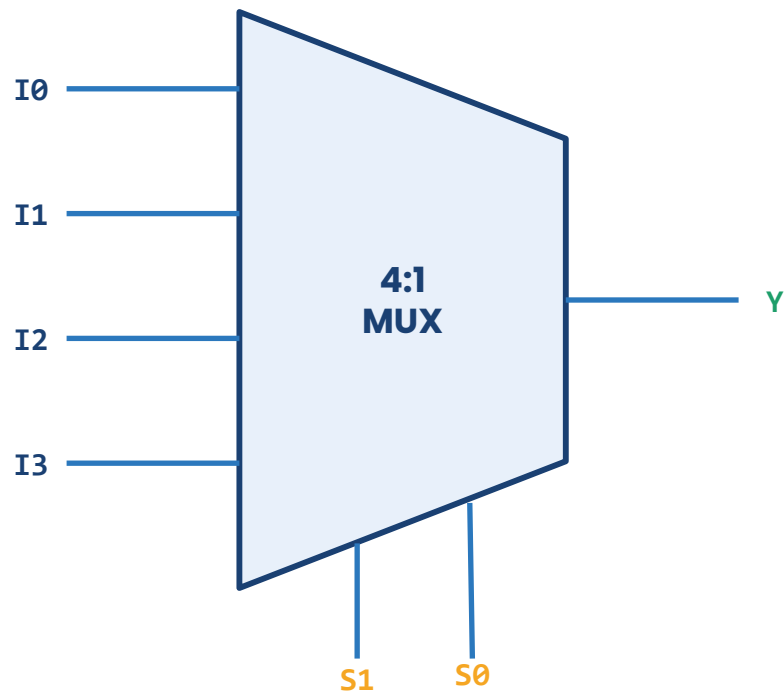


READING IT

- The top AND passes A only when $S'=1$ (i.e. $S=0$).
- The bottom AND passes B only when $S=1$.
- At any moment exactly one AND is 'open'; the OR merges them into Y.

The 4:1 Multiplexer

Two select lines address four inputs — 2 select bits choose 1 of $2^2 = 4$.



SELECT → OUTPUT

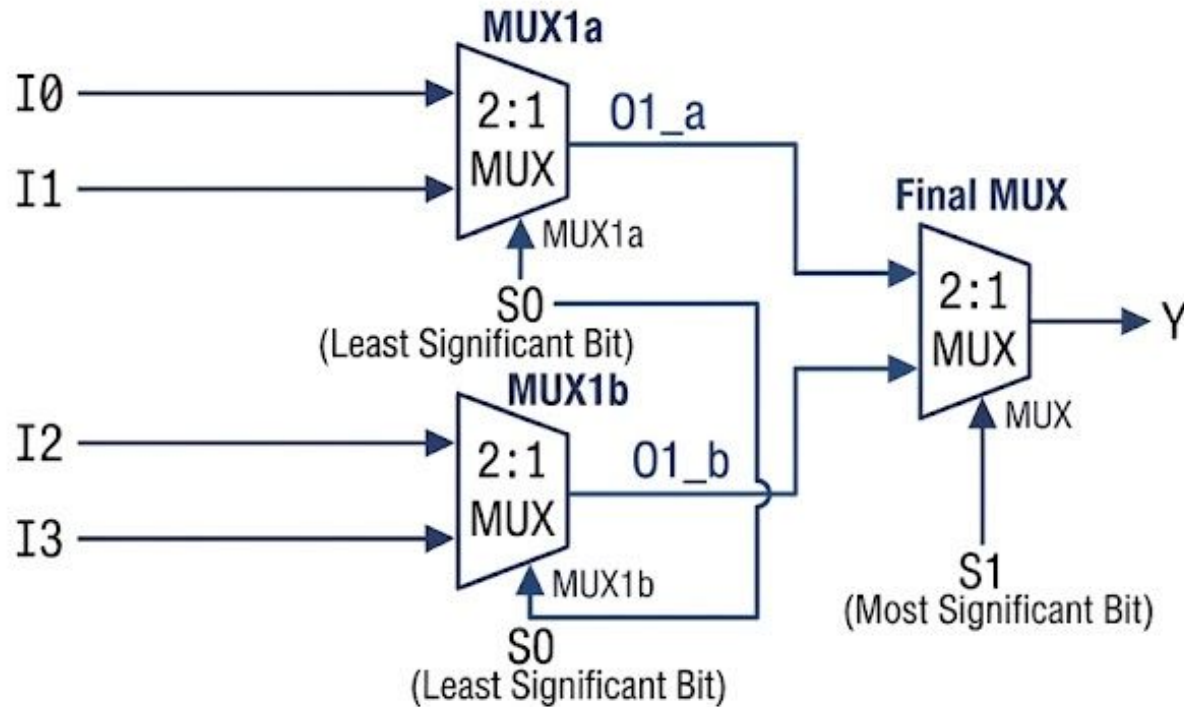
S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

$$Y = S1'S0' \cdot I0 + S1'S0 \cdot I1 + S1S0' \cdot I2 + S1S0 \cdot I3$$

Each select combination = one minterm that gates exactly one input through.

Cascading: Build Big MUXes from Small Ones

A 4:1 MUX is just three 2:1 MUXes in a tree.



WHY IT MATTERS

One small, verified block scales to any size.

**Need an 8:1? Two 4:1s feeding a 2:1.
Need 16:1? Keep tree-ing.**

This 'compose small proven parts into big ones' is exactly how real chips — and good software — are built.

A MUX Can Be Any Logic Function

Wire the inputs to constants, and a 4:1 MUX implements ANY 2-variable function.

THE TRICK

Feed the input variables into the SELECT lines. Then wire each data input I_k to the 0 or 1 that the truth table demands for that row.

Example — implement XOR(A,B):

S1=A, S0=B (selects)

I0 = 0 (A=0, B=0 → XOR=0)

I1 = 1 (A=0, B=1 → XOR=1)

I2 = 1 (A=1, B=0 → XOR=1)

I3 = 0 (A=1, B=1 → XOR=0)

WHY THIS IS A BIG DEAL

A $2^n:1$ MUX implements ANY n -variable function — no minimisation needed.

This is the principle behind the Lookup Tables (LUTs) inside every FPGA.

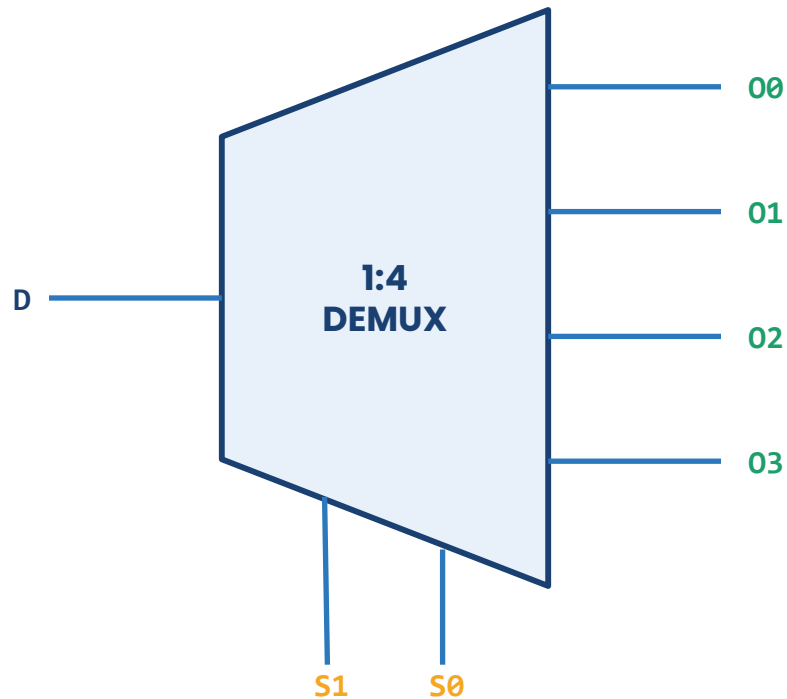
An FPGA is, at its core, a sea of tiny MUX-based LUTs you reprogram by changing what the data inputs are wired to — which is why an FPGA can 'become' any circuit.

2. Demultiplexers & Decoders

The reverse of a MUX — route one input to many outputs, or light up one line from a code.

The Demultiplexer (DEMUX)

One input, many outputs — the select code decides which output it reaches.



MUX RUN BACKWARDS

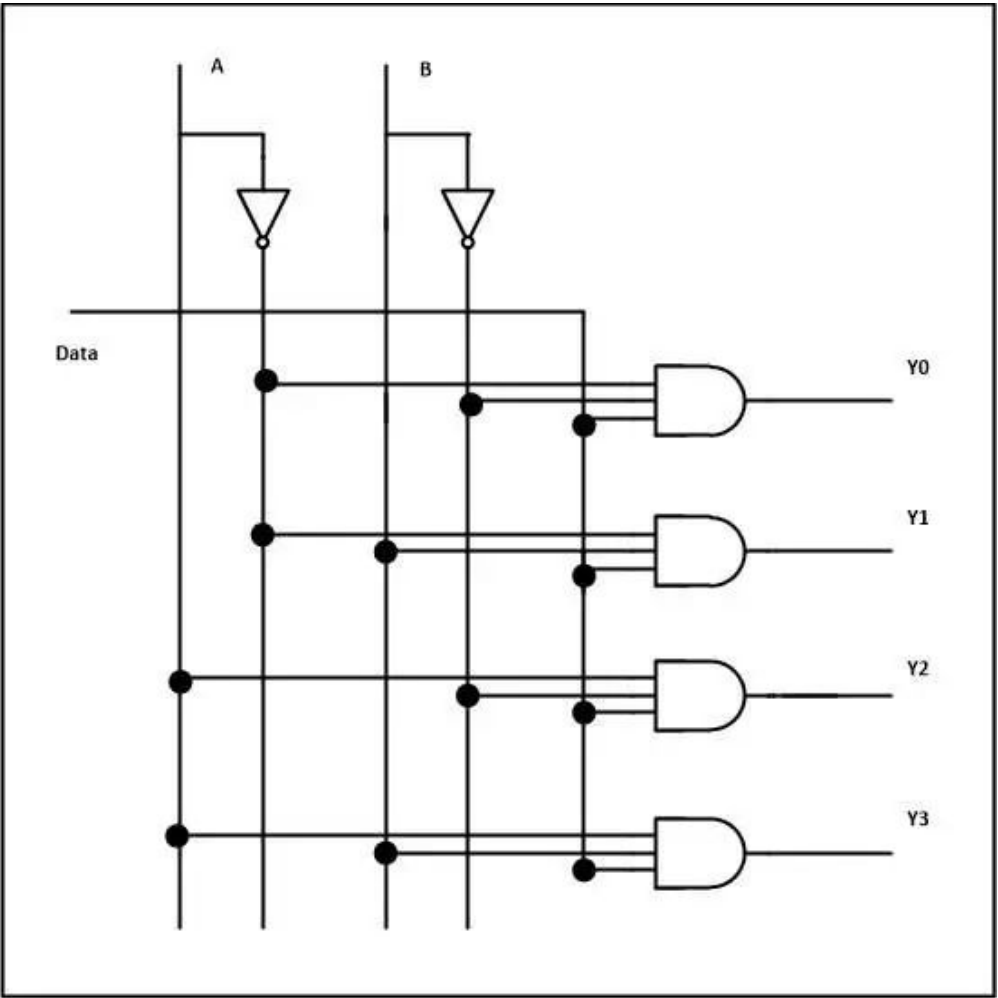
A MUX has many inputs → one output. A DEMUX flips it: one input → many outputs. The select bits choose the single destination; all other outputs stay at 0.

REAL USE

A single data line fanned out to one of many destinations — e.g. a CPU writing a result to one chosen register, or a network switch routing a packet to one output port.

The Demultiplexer (DEMUX)

One input, many outputs — the select code decides which output it reaches.



ROUTING TABLE

A	B	Y0	Y1	Y2	Y3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D